

2026.02.27.금

🕒 생성일	@2026년 2월 27일 오전 9:11
🏷️ 태그	AWS Kubernetes

목차

1. Container K8s 이론 복습
2. EKS로 2048 배포

▼ 1) 이전 TIL 내용 참고

- 2026.02.19 ~
- 2026.02.26

▼ 2) EKS로 2048 배포

- AWS EC2 t3.micro ubuntu 생성
- EKS 설치

다음 명령어로 kubectl을 다운로드 합니다.

Copy
sudo curl -o /usr/local/bin/kubectl https://s3.us-west-2.amazonaws.com/amazon-eks/1.26.4/2023-05-11/bin/linux/amd64/kubectl
다음 명령어로 실행 권한을 줍니다.

Copy
sudo chmod +x /usr/local/bin/kubectl
잘 설치가 되었는지 버전 확인을 해봅니다.

Copy
kubectl version --client=true --short=true
다음 명령어로 eksctl을 다운로드하고 압축을 풉니다.

Copy
curl --location "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_\${uname -s}_amd64.tar.gz" | tar xz -C /tmp
다음 명령어로 eksctl을 /usr/local/bin 디렉터리로 이동합니다.

Copy
sudo mv -v /tmp/eksctl /usr/local/bin
다음 명령어로 설치가 잘되었는지 확인합니다.

Copy
eksctl version
다음 명령어로 현재 리전의 정보를 환경변수에 저장한다.

Copy
export AWS_REGION=\$(curl --silent http://169.254.169.254/latest/meta-data/placement/region) && echo

`$AWS_REGION`

다음 명령어로 eks를 배포합니다. 15분 정도 걸립니다.

Copy

```
eksctl create cluster --name myeks --version 1.26 --region ${AWS_REGION}
```

다음 명령어로 노드의 리스트를 확인합니다.

Copy

```
kubectl get nodes
```

아래처럼 노드의 리스트를 확인할 수 있습니다.

Copy

```
ip-192-168-51-108.ap-northeast-2.compute.internal Ready <none> 115s v1.23.17-eks-0a21954
ip-192-168-69-32.ap-northeast-2.compute.internal Ready <none> 100s v1.23.17-eks-0a21954
```

EKS를 설치 하고 EKS에 명령을 날리기 위한 유틸리티 kubectl을 설치 완료하였습니다.

- Pod 배포

```
cat <<EOF > nginx-orange.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-orange
  labels:
    run: nginx-orange
spec:
  containers:
    - name: orange
      image: nginx
EOF
```

```
kubectl apply -f nginx-orange.yaml
```

```
kubectl get pods -o wide
```

```
kubectl delete pods nginx-apple
```

- ReplicaSet 배포

```
cat <<EOF > replicaset.yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-replicaset
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-replicaset
  template:
    metadata:
      labels:
        app: nginx-replicaset
    spec:
      containers:
        - name: nginx-container
```

```
        image: nginx
EOF
```

```
kubectl apply -f replicaset.yaml

kubectl get replicaset,pods

kubectl scale replicaset nginx-replicaset --replicas=5

kubectl get replicaset,pods

kubectl delete replicaset nginx-replicaset
```

- Deployment 배포

```
cat <<EOF > deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deploy
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx-deploy
  template:
    metadata:
      labels:
        app: nginx-deploy
    spec:
      containers:
        - image: nginx
          name: nginx-container
EOF
```

```
kubectl apply -f deployment.yaml

kubectl get deployment,replicaset,pods
```

- Service 배포

```
cat <<EOF > 2048.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: game-2048
  labels:
    run: game-2048
spec:
  replicas: 3
  selector:
    matchLabels:
      app: game-2048
  template:
    metadata:
      labels:
        app: game-2048
    spec:
```

```

    containers:
      - image: public.ecr.aws/kishorj/docker-2048:latest
        name: game-2048
        ports:
          - containerPort: 80
            protocol: TCP
    ---
apiVersion: v1
kind: Service
metadata:
  name: "game-2048"
spec:
  type: LoadBalancer
  ports:
    - port: 80
      protocol: TCP
      targetPort: 80
  selector:
    app: game-2048
EOF

```

```

kubectl apply -f 2048.yaml

kubectl get deploy,svc

kubectl delete -f 2048.yaml

```

- Namespace 배포

```

cat <<EOF > mynamespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: "mynamespace"
---
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  labels:
    run: nginx
  namespace: mynamespace
spec:
  containers:
    - name: orange
      image: nginx
EOF

```

```

kubectl apply -f mynamespace.yaml

kubectl get pod -n mynamespace

kubectl delete namespace mynamespace

```

▼ 3) Github Actions - S3

1. 깃 repo 생성
2. 로컬 폴더에 깃 설치, 레포 연결

3. 생성한 레포 설정 → settings → secrets and variables에 AWS 아마존 루트 계정 액세스 키, 시크릿 키 변수 등록
4. 로컬 폴더에 디렉토리 생성

```
mkdir .github
mkdir .github/workflows

cd .github/workflows
```

5. 깃허브 액션 yaml 파일 작성

```
name: Sync to S3

on:
  push:
    branches:
      - main

jobs:
  sync:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v3

      - name: Configure AWS Credentials
        uses: aws-actions/configure-aws-credentials@v2
        with:
          aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
          aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
          aws-region: ap-northeast-2 # Change this to your region of your bucket

      - name: Deploy to S3
        run: |
          echo "Syncing specific files to S3..."
          aws s3 cp . s3://버킷명/폴더명/ \
            --recursive \
            --exclude "*" \
            --include "*.html" \
            --include "*.css" \
            --include "*.js"
          echo "Deployment complete."
```

6. S3 생성 → 정적 웹 사이트 배포 설정
7. index.html, style.css, script.js 등 파일 루트에 작성 및 깃허브 푸시
8. 깃허브 Actions 동작 확인
9. AWS S3 > 객체 > 작업 > ACL 사용해서 퍼블릭 설정 (권한)
 - 원래 미리 권한을 부여했어야 한다.
10. AWS S3 엔드포인트 URL/index.html 으로 확인 가능
11. 파일 수정하고 깃허브 푸시, 권한 부여(9번 반복)하면 수정 사항 확인 가능